CS6008 – Network Security

Building a Secure Chat Application

Complete Report – Phases 1 to 5

23B1023 & 23B0945

September 2026

1 Introduction

This report documents the design, implementation, and verification of a secure one-to-one chat application, built in five progressive phases. Each phase adds exactly one security property on top of the previous phase, and after every phase that introduces a cryptographic protection, we attempt to break the previous phase using an attack tool built ourselves, showing with evidence whether the new phase actually stops that attack.

- **Phase 1:** A baseline TCP chat application with no security – all messages travel in plaintext, and the server can read everything it relays.
- **Phase 2:** Client–server confidentiality using a hand-implemented Diffie–Hellman key exchange and AES-256-GCM encryption, followed by a Man-in-the-Middle (MITM) attack that shows key exchange alone does not provide authentication.
- **Phase 3:** Server authentication via a minimal PKI (Certificate Authority, server certificate, and proof-of-possession), which defeats the Phase 2 MITM attack.
- **Phase 4:** End-to-end encryption directly between the two clients, so that even the server cannot read message content.
- **Phase 5:** Forward secrecy through automatic key rotation every 60 seconds, limiting the damage from any single key compromise.

The application is written in C++ using POSIX socket APIs (`socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`) and C++ standard threading (`std::thread`, `std::mutex`, `std::atomic`). All cryptographic primitives that are permitted to use library support (AES-GCM, SHA-256, X.509 parsing) are implemented through OpenSSL's low-level EVP/BN/X509 interfaces; the Diffie–Hellman key exchange itself, including the modular exponentiation, is implemented from scratch in every phase that uses it, and no TLS/SSL or built-in DH library API is used anywhere.

2 VM Topology

The application is deployed across separate virtual machines on the same virtual network (VirtualBox Host-Only Network), rather than solely over loopback, so that the Wireshark captures and the MITM demonstrations reflect a real network setup. Basic connectivity between all VMs was verified using `ping` before running the chat application. From Phase

2 onward, a fourth VM (Mallory) is added on the same network to host the MITM proxy used in the attack tasks.

Role	VM Name	IP Address
Server (S)	Server VM	`192.168.100.5`
Client 1 (C1 – Alice)	Client VM 1	`192.168.100.6`
Client 2 (C2 – Bob)	Client VM 2	`192.168.100.4`
Attacker (Mallory, from Phase 2 onward)	Mallory VM	`192.168.100.7`

Table 1: VM topology used throughout the assignment.

3 Phase 1: Baseline Chat Application (No Security)

In this phase we built a basic one-to-one TCP chat application consisting of a server and a client program, with **no encryption or security**: all messages are transmitted in plaintext over the network. The goal is to establish a working baseline that is progressively secured in the later phases.

The server acts as a central relay: it accepts connections from clients, stores their usernames, and forwards messages between them over raw TCP sockets. The server can read every message it relays.

3.1 Protocol Design

3.1.1 Overview

Our chat protocol is a simple, text-based, newline-delimited protocol running over TCP. Each message between client and server is a single line of text terminated by a newline character (`\n`), which makes the protocol easy to parse and debug.

3.1.2 Message Framing

We use **newline-delimited framing**. Each protocol message is a single line ending with `\n`. On the receiving side (both client and server), incoming bytes are buffered until a complete newline-terminated line is found, which handles the case where TCP may deliver data in arbitrary chunks – we keep appending to a buffer and only process complete lines.

Specifically, both the client and server maintain a `pending_data` string buffer. When bytes arrive via `recv()`, they are appended to this buffer. We then scan for `\n` characters; each complete line (everything before the newline) is extracted and processed as one protocol message, and the processed portion is removed from the buffer.

3.1.3 Protocol Commands

The protocol uses a simple tag-based format: each line starts with a command tag followed by any relevant data.

Direction	Command	Description
Client → Server	`LOGIN <username>`	Register with the server using the given username.
Client → Server	`MSG <recipient> <message>`	Send a message to the specified recipient.
Client → Server	`WHO`	Request the list of currently online users.
Client → Server	`QUIT`	Disconnect cleanly from the server.
Server → Client	`OK <info>`	Confirmation response (e.g., login success, message delivered).
Server → Client	`ERR <reason>`	Error response with a description.
Server → Client	`FROM <sender> <message>`	A message relayed from another user.
Server → Client	`USERS <user1> <user2> ...`	List of currently online users.

Table 2: Protocol commands and their descriptions.

3.1.4 Message Routing

The server maintains a map (`std::map<std::string, int>`) that maps each logged-in username to its corresponding socket file descriptor. When a client sends a `MSG` command targeting a recipient, the server looks up the recipient's socket in this map and forwards the message using the `FROM` tag. If the recipient is not online, the server replies to the sender with an `ERR` message.

This design naturally supports more than two clients: any number of clients can connect, log in with unique usernames, and send messages to any other online user, with the server routing messages based on the username specified in the `MSG` command.

3.1.5 Client Command Interface

The client provides the following user-facing commands at the prompt:

- `@username message` — Sends a message to `username` and selects them as the current chat partner.
- `/chat username` — Switches the selected chat partner without sending a message.
- `/who` — Shows the list of currently online users.
- `/quit` — Disconnects from the server and exits.

Any other text typed at the prompt is sent as a message to the currently selected chat partner. If no partner is selected, the client prompts the user to select one first.

3.2 Implementation Details

3.2.1 Source Files

- `protocol.h` — Shared header defining constants (e.g., `BUFFER_SIZE`, `CHAT_PORT`) and helper functions (e.g., `starts_with`).
- `server.cpp` — The chat server implementation.
- `client.cpp` — The chat client implementation.

3.2.2 Server Architecture

- The server listens on a configurable TCP port (default defined in `protocol.h`).
- For each incoming connection, it spawns a new thread (`std::thread`) that runs `handle_client()`. The thread is detached so the server can continue accepting new connections.
- A global `clients` map (protected by `clients_mutex`) stores the mapping from usernames to socket descriptors.
- Each socket also has a per-socket write mutex (`client_write_mutexes`) to prevent interleaved writes when multiple threads try to send to the same client at the same time.
- When a client disconnects (either by sending `QUIT` or by closing the connection), the server removes it from the map and closes the socket.

3.2.3 Client Architecture

- The client connects to the server, sends a `LOGIN` message with its username, and then spawns a background receiver thread that continuously reads incoming messages and prints them.
- The main thread reads user input from `stdin` and translates it into protocol commands.
- The client maintains a `selected_user` variable to track the currently selected chat partner.
- An `std::atomic<bool>` flag (`running`) is shared between the main thread and the receiver thread to coordinate shutdown.

3.2.4 Reliable Sending

Both client and server use a `send_all()` function that loops over `send()` until all bytes have been transmitted, handling the case where a single `send()` call may not transmit all the data in one shot.

3.3 Verification

3.3.1 Server Plaintext Logging

The server is instrumented to log every message it relays to `stdout`. Specifically, in the `handle_message()` function, before forwarding a message to the recipient, the server prints:

```
std::cout << "[RELAY PLAINTEXT] "
          << sender << " -> "
          << recipient << ": "
          << message
          << std::endl;
```

This confirms that the server has full access to the plaintext content of every message passing through it. There is no encryption or obfuscation of any kind in this phase.

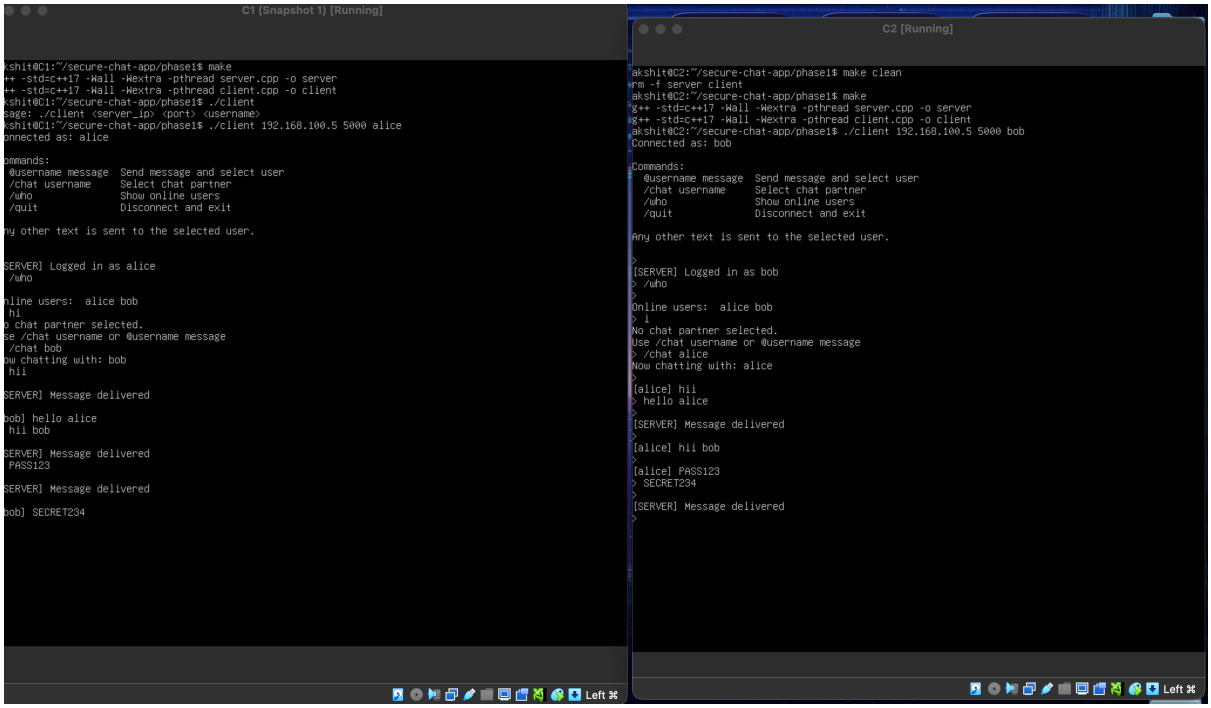

Figure 1: Alice and Bob exchanging messages through the chat application.

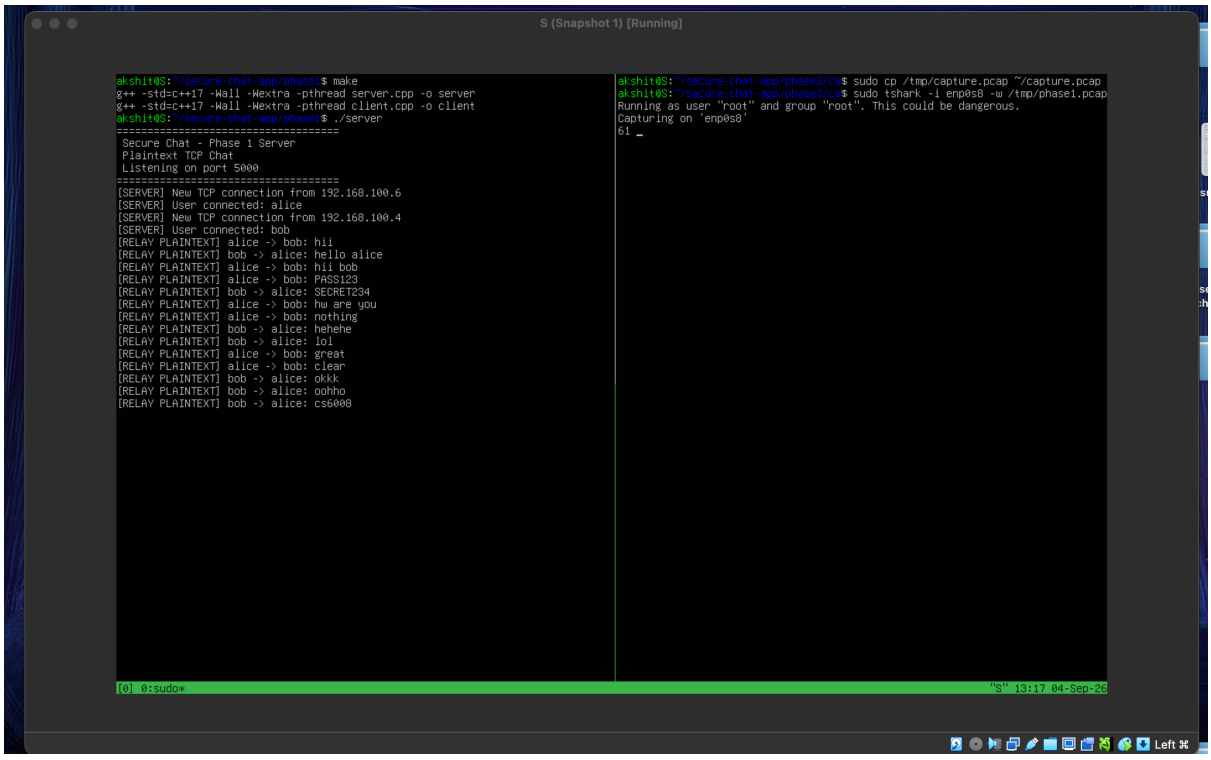

Figure 2: Server terminal showing `[RELAY PLAINTEXT]` logs for all relayed messages.

3.3.2 Wireshark Traffic Capture

We captured live TCP traffic between a client and the server using Wireshark on the relevant network interface. We filtered on the chat port and used **Follow** → **TCP Stream** to view the raw data exchanged.

The capture clearly shows that all protocol messages – including `LOGIN`, `MSG`, `FROM`, and `OK` – are transmitted in plaintext. The message content is fully readable in the raw TCP stream, confirming that there is no encryption in Phase 1. This baseline capture serves as the reference point for Phase 2, where the same traffic becomes encrypted after adding Diffie–Hellman key exchange and AES-GCM encryption.

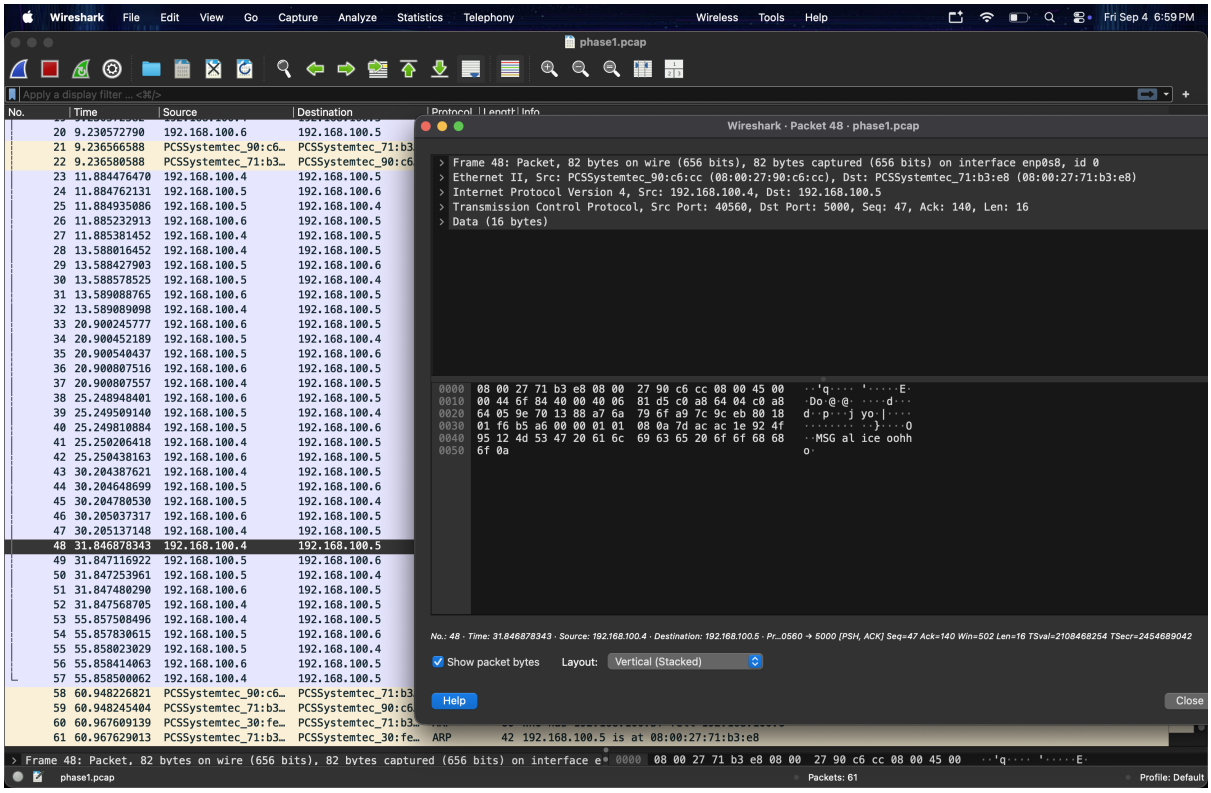

Figure 3: Wireshark "Follow TCP Stream" showing all chat messages in plaintext.

3.4 Conclusion

Phase 1 delivers a fully functional TCP-based chat application with a simple text-based protocol. The server correctly relays messages between clients, and the client supports all required commands (`@username`, `/chat`, `/who`, `/quit`). We verified that the server can read every message in plaintext (via its relay logs) and that network traffic captured in Wireshark is also fully readable – there is currently **no confidentiality**: anyone who can observe the network traffic, or the server itself, can read all messages. This is addressed in the phases that follow.

4 Phase 2: Client-Server Confidentiality via Diffie–Hellman

Phase 2 fixes the plaintext problem from Phase 1 by adding **Diffie–Hellman (DH) key exchange** and **AES-256-GCM encryption** on every client–server link. Each client independently performs a DH handshake with the server when it connects, and all subsequent traffic on that link (including the `LOGIN` command) is encrypted.

We also built a **Man-in-the-Middle (MITM) proxy** to demonstrate that DH alone does not authenticate who you are talking to – an attacker positioned between a client and the server can intercept everything despite the encryption.

4.1 Diffie–Hellman Key Exchange

We implemented the DH key exchange from scratch using OpenSSL's BIGNUM library (`openssl/bn.h`) for arbitrary-precision arithmetic. We did **not** use any library-provided DH class or API.

- **Prime group:** We use the 2048-bit MODP Group 14 prime from RFC 3526, with generator $g = 2$ – a well-known, published safe prime; we do not generate our own.
- **Key generation:** Each side picks a random 2048-bit private exponent a, then computes the public value $A = g^a \bmod p$ using `BN_mod_exp`.
- **Shared secret:** On receiving the other side's public value B, we compute $s = B^a \bmod p$. Both sides end up with the same s.
- **Validation:** Before computing the shared secret, we reject peer public values that are ≤ 1 or $\geq p - 1$. This prevents small-subgroup attacks where an attacker sends values like 0, 1, or $p - 1$ to force a predictable shared secret.

4.1.1 Handshake Flow

1. Client generates its DH keypair and sends: `DH_INIT <public_value_hex>`
2. Server generates its own DH keypair, computes the shared secret from the client's public value, and replies: `DH_ACK <public_value_hex>`
3. Client computes the shared secret from the server's public value.
4. Both sides now hold the same raw shared secret.

Each client performs an **independent** DH exchange with the server, so the C1–S link and the C2–S link have different session keys.

4.2 Why Hash the Raw Shared Secret?

The raw DH shared secret is a 2048-bit number and cannot be used directly as an AES key:

- AES-256 needs exactly 256 bits, not 2048.
- The raw secret is not uniformly distributed across all 256-bit values – its bits have mathematical structure tied to the DH group. A hash function (SHA-256) acts as a randomness extractor that produces a fixed-size, uniformly distributed key from the structured input.
- If even a few bits of the raw secret were partially predictable, using it directly would leak that structure into the key. Hashing destroys any such patterns.

We use `SHA-256` (via OpenSSL's EVP interface) to derive the 256-bit AES key from the raw shared secret, in `crypto_utils.cpp` using the `derive_key()` function.

4.3 AES-256-GCM Encryption

After the DH handshake, every message on the wire is encrypted using AES-256-GCM. We chose GCM (Galois/Counter Mode) because it provides **authenticated encryption** – it encrypts the message *and* produces an authentication tag that detects any tampering.

4.3.1 Why AES-GCM and not Plain AES (e.g., AES-CBC)?

Plain AES modes like CBC only provide confidentiality. An attacker who flips a bit in the ciphertext will get corrupted plaintext on the other side, but the receiver has no way to detect that corruption happened. With GCM, any modification to the ciphertext (or the nonce, or the tag) causes decryption to fail outright with an authentication error, rather than silently producing garbage.

4.3.2 Encryption Details

Each encrypted message on the wire is a base64-encoded blob with the following structure:

```
nonce (12 bytes) || ciphertext || tag (16 bytes)
```

- A fresh 12-byte random nonce is generated for every message using `RAND_bytes`.
- The 16-byte GCM authentication tag is appended after the ciphertext.
- The entire blob is base64-encoded before sending, so it can safely travel as a single newline-delimited line (raw ciphertext could contain `0x0A` bytes that would break the framing).

4.4 Source Files

File	Purpose
`protocol.h`	Shared constants and helpers (same as Phase 1).
`dh.h / dh.cpp`	DiffieHellman class — keypair generation, public value export, shared secret computation. Uses RFC 3526 Group 14.
`crypto_utils.h / .cpp`	`derive_key()` (SHA-256 key derivation) and `fingerprint()` (truncated hash for verification).
`aes_gcm.h / .cpp`	`encrypt_message()` and `decrypt_message()` using AES-256-GCM via OpenSSL's EVP interface.
`base64.h / .cpp`	Base64 encode/decode wrappers around OpenSSL's `EVP_EncodeBlock / EVP_DecodeBlock`.
`openssl_raii.h`	RAII wrappers (`unique_ptr` with custom deleters) for OpenSSL handles.
`server.cpp`	Updated server with DH handshake and encrypted send/receive.
`client.cpp`	Updated client with DH handshake and encrypted send/receive.
`mitm.cpp`	Man-in-the-Middle proxy for the attack task.

Table 3: Phase 2 source files.

4.5 Verification

4.5.1 Fingerprint Matching

After the DH handshake, both sides compute a fingerprint of the shared secret: the first 8 bytes of `SHA-256(raw_secret)`, displayed as colon-separated hex (e.g., `3f:a1:9c:02:77:8e:5b:d4`). This fingerprint is printed on both the client and server terminals, and we verified that both sides show the **identical fingerprint** for a given connection. We never print the raw secret or the derived AES key – only this truncated hash.

4.5.2 Encrypted Communication (Before MITM)

After the handshake, Alice and Bob exchange messages normally. The server still relays messages, but now all traffic on the wire is encrypted.

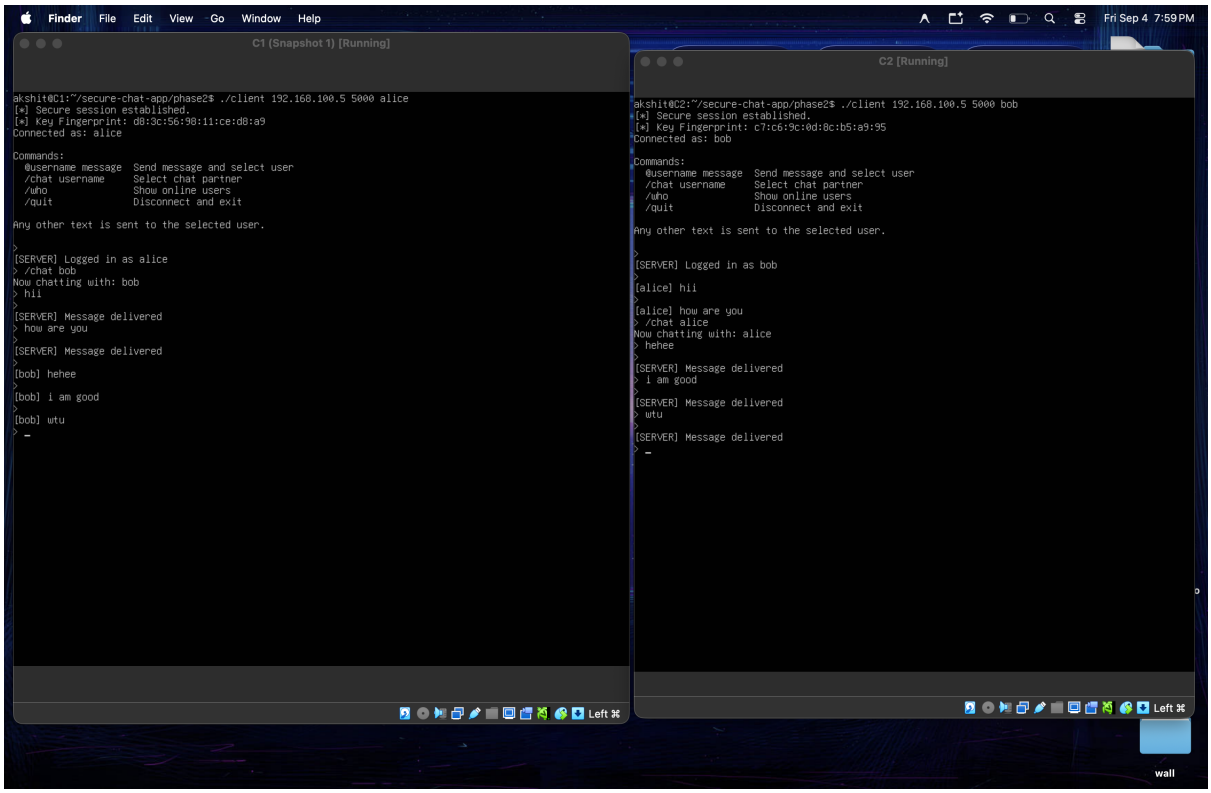

Figure 4: Alice and Bob terminals showing matching key fingerprints and encrypted chat.

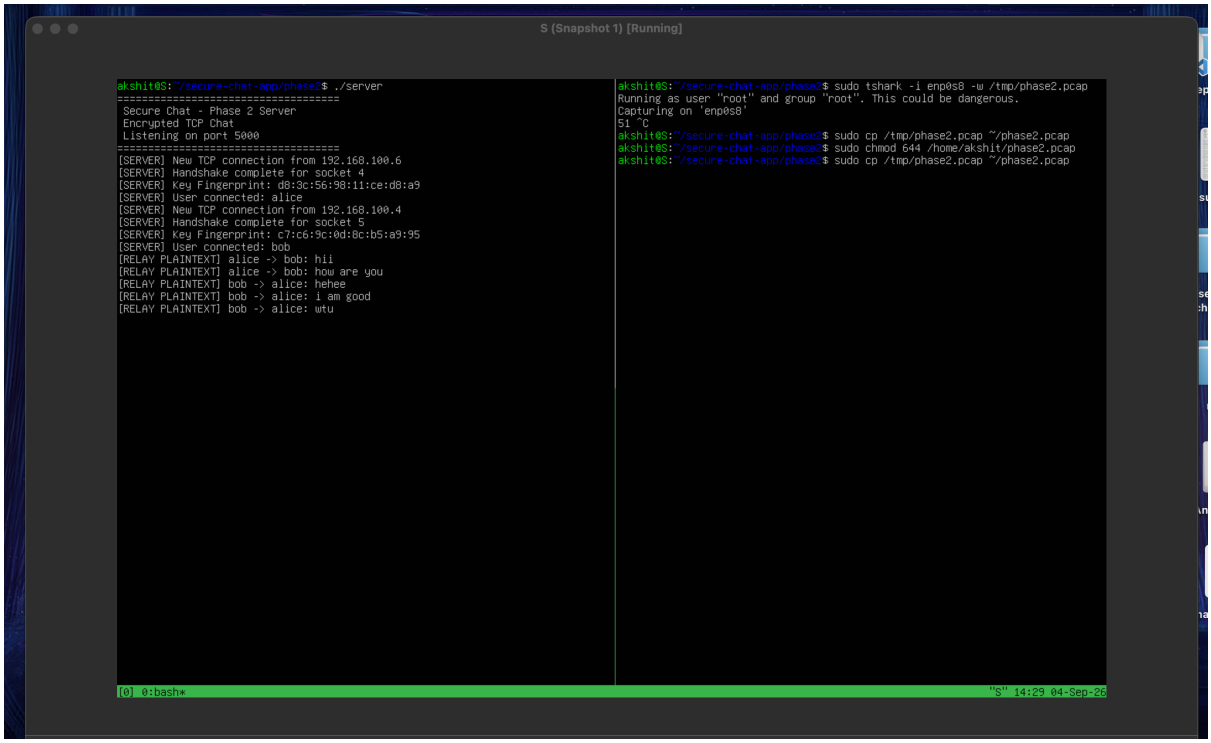

Figure 5: Server terminal — the server still sees plaintext internally (after decryption) and logs relayed messages.

4.5.3 Wireshark Capture and Tamper Detection

We captured traffic on the same interface and port as in Phase 1. Using "Follow → TCP Stream," the content now shows base64-encoded ciphertext instead of readable plaintext, confirming that the DH + AES-GCM encryption is working – a passive network observer can no longer read the messages.

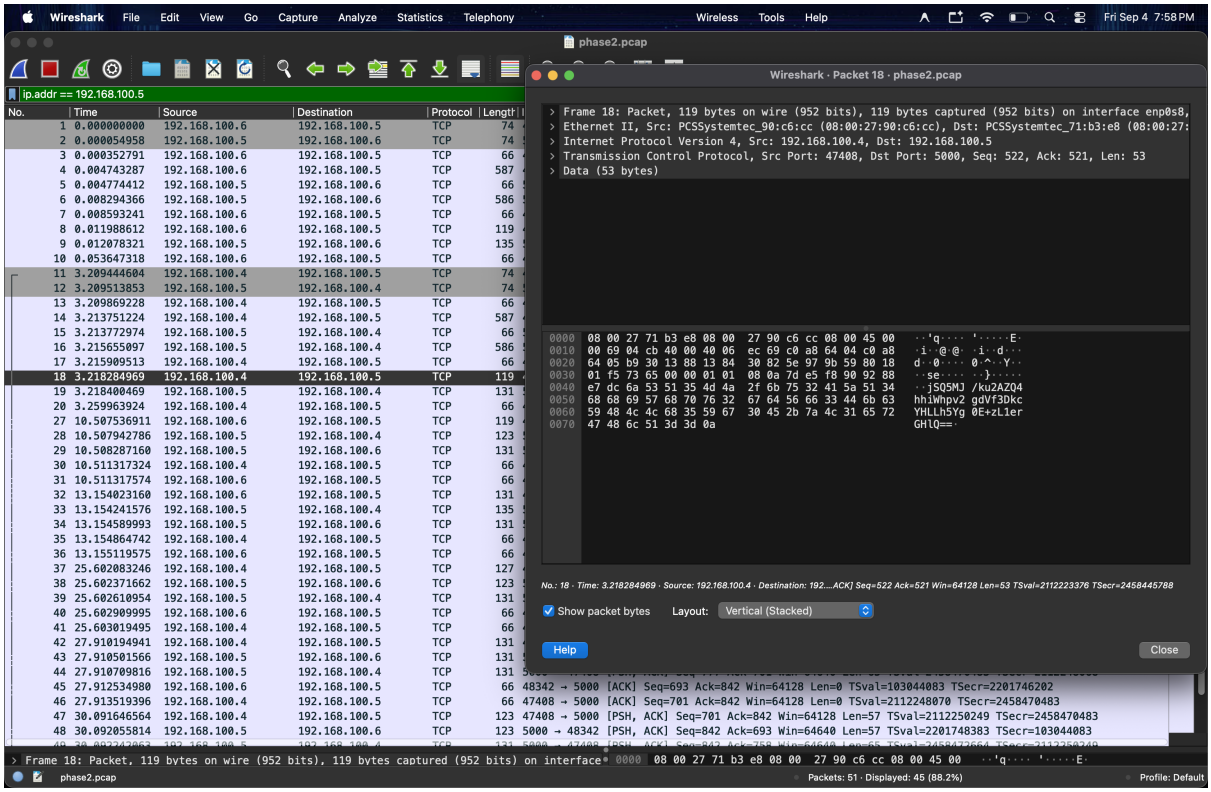

Figure 6: Wireshark TCP stream showing encrypted (base64-encoded) traffic — contrast with Phase 1 where messages were readable.

To verify that GCM detects tampering, we built a test directly into the MITM proxy: when a message contains the string `TRIGGER_TAMPER`, the proxy flips a single byte in the ciphertext before forwarding it. On the receiving side, decryption fails with:

```
[CRYPTO ERROR] Tampering detected: GCM authentication failed
    -- message was tampered with or corrupted
```

This confirms that AES-GCM rejects modified ciphertext rather than silently producing corrupted plaintext, which is exactly the property that distinguishes authenticated encryption from plain AES.

4.6 Attack Task: Man-in-the-Middle

4.6.1 Why DH Alone Is Not Enough

Diffie–Hellman establishes a shared secret with *whoever answers the connection*. It does not verify *who* that is. If an attacker (Mallory) sits between the client and server, she can perform two independent DH exchanges – one with the client (pretending to be the server) and one with the real server (pretending to be the client). She ends up with two separate session keys and can decrypt, read, and re-encrypt every message passing through.

4.6.2 MITM Proxy Implementation

We built a separate program (`mitm.cpp`) that works as follows:

1. Mallory listens on port 5001 and waits for the victim client to connect.
2. When the client connects and sends `DH_INIT`, Mallory does **not** forward it to the real server. Instead, she completes a DH exchange with the client using her own keypair, obtaining `key_client`.
3. Mallory then opens a connection to the real server and performs a *separate* DH exchange with it, obtaining `key_server`.
4. For every subsequent message: Mallory decrypts with the sender's key, logs the plaintext, re-encrypts with the receiver's key, and forwards it.

The victim client is pointed at Mallory's IP/port instead of the real server's. Both the client and the server complete their DH handshakes successfully and believe they are communicating directly.

4.6.3 MITM Attack Results

We ran the proxy on the Mallory VM. The client was configured to connect to Mallory's address (`192.168.100.7:5001`) instead of the real server's.

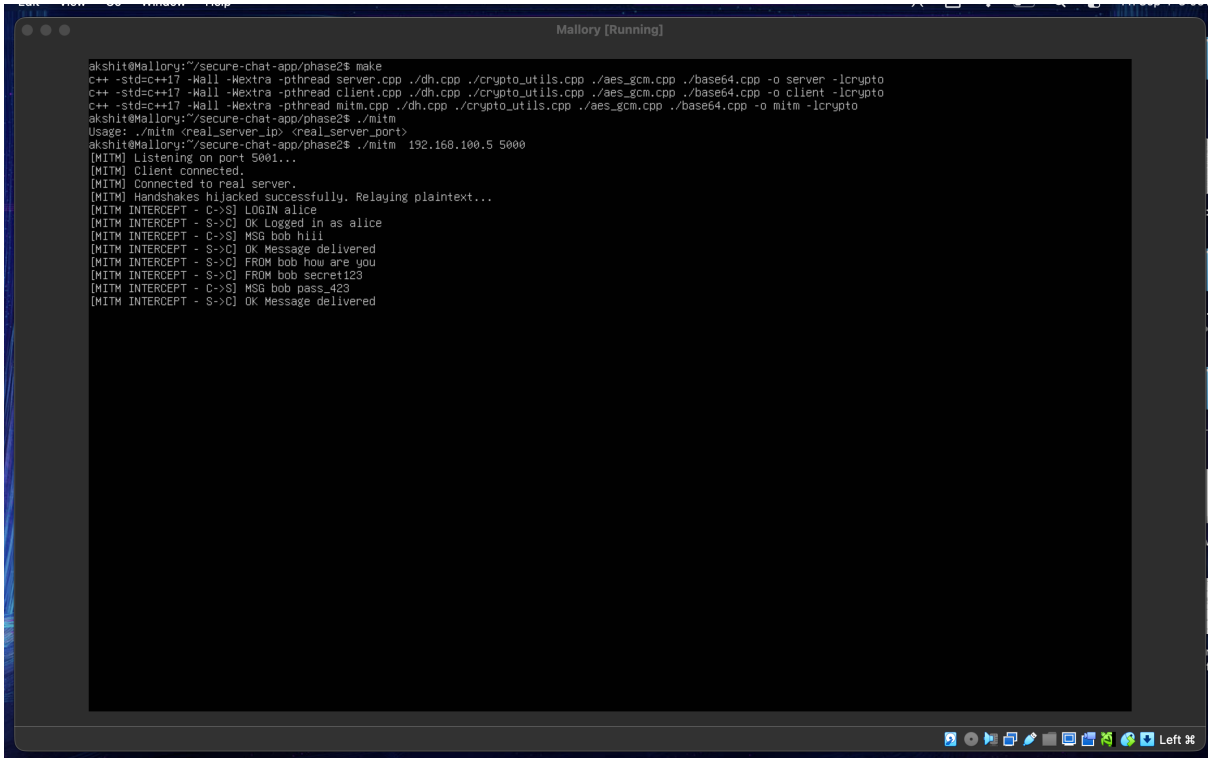

Figure 7: Mallory's terminal showing intercepted plaintext via `[MITM INTERCEPT]` logs.

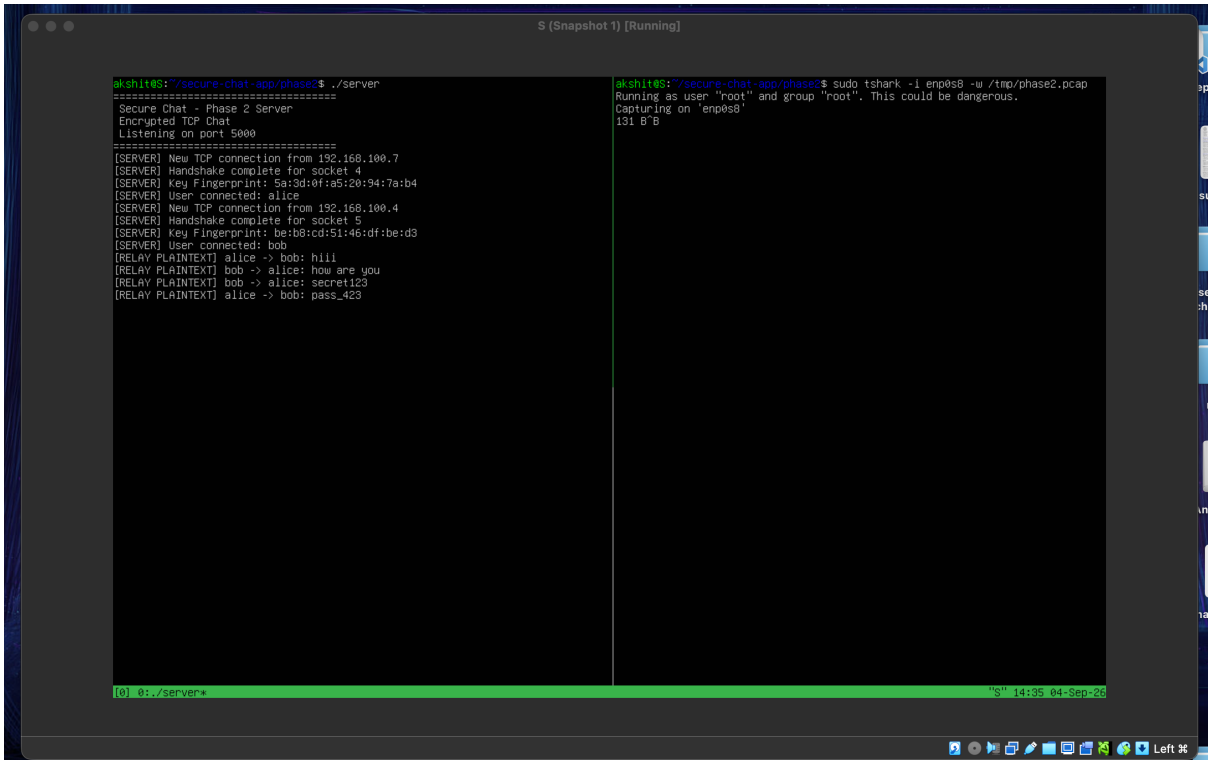

Figure 8: Server terminal during MITM attack — the server sees nothing unusual.

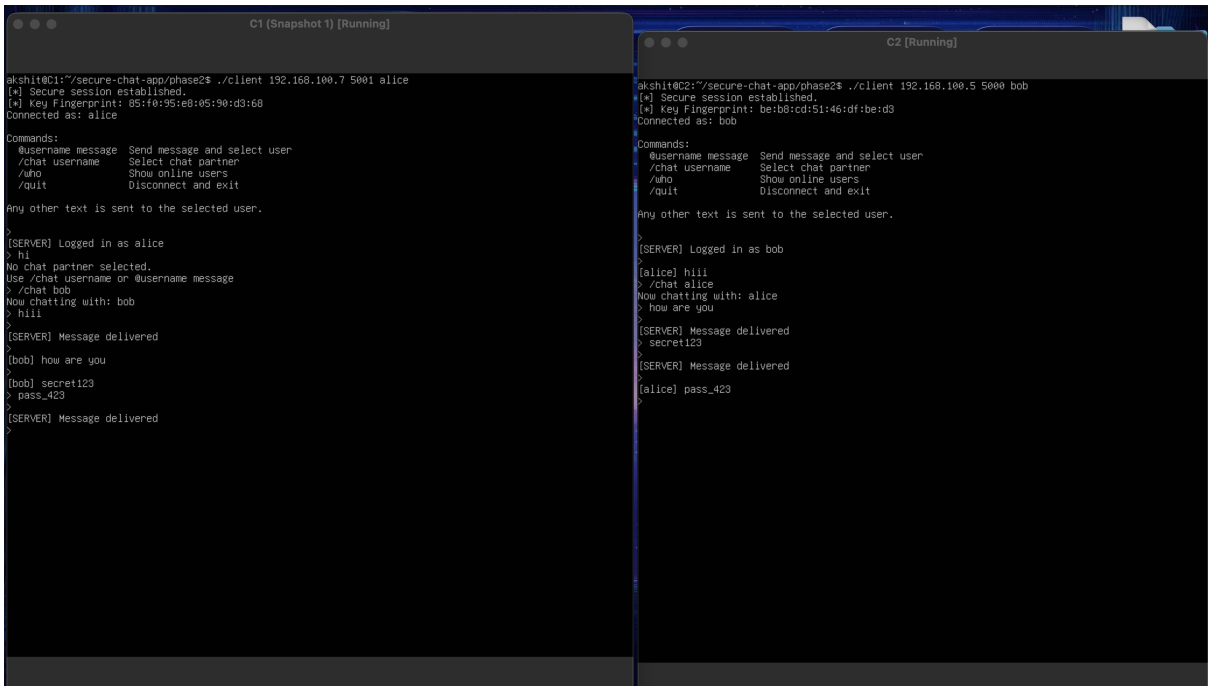

Figure 9: Alice and Bob terminals during MITM attack — chat works normally, neither side notices the interception.

4.6.4 Wireshark During MITM

We also captured traffic during the MITM attack to show the two separate encrypted links (client ↔ Mallory and Mallory ↔ server).

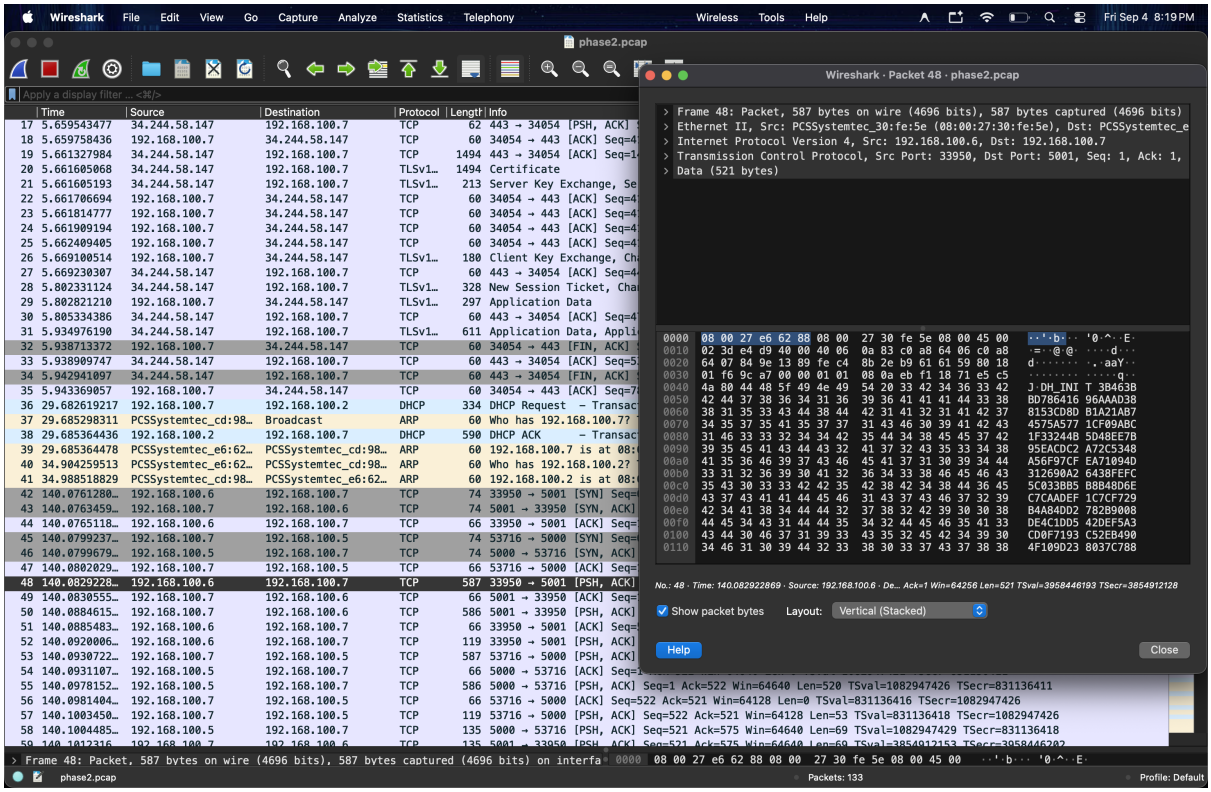

Figure 10: Wireshark capture during MITM attack.

4.6.5 Could the Victim Detect the Attack?

In theory, the victim could notice something if they manually compared fingerprints out-of-band. The client's fingerprint would match Mallory's key (not the server's), and the server's fingerprint would also match Mallory's other key (not the client's). If the client and server compared fingerprints over a separate trusted channel (e.g., a phone call), they would see a mismatch.

In practice, an ordinary user would **not** notice anything. The chat works perfectly, there are no errors or warnings, and there is no built-in mechanism to verify who you performed the DH exchange with. This is exactly the gap that Phase 3 (server authentication via PKI) addresses.

4.7 Conclusion

In Phase 2, we added DH key exchange and AES-256-GCM encryption to every client–server link. Wireshark confirms that the traffic is no longer readable, and AES-GCM correctly rejects tampered ciphertext. However, we also demonstrated that DH by itself does not provide authentication: our MITM proxy successfully intercepted all messages by performing two independent handshakes, and neither the client nor the server could detect the attack. This motivates Phase 3, where we add server authentication using certificates so the client can verify it is actually talking to the real server before proceeding with the key exchange.

5 Phase 3: Server Authentication via PKI

Phase 2 showed that a raw Diffie–Hellman key exchange, while providing confidentiality, is vulnerable to a MITM attack because the client has no way to verify *who* it is exchanging keys with. Phase 3 addresses this by introducing a **Public Key Infrastructure (PKI)**. Before any DH exchange takes place, the server must now prove its identity to the client through a CA-signed certificate and a proof-of-possession challenge. If verification fails, the client aborts immediately without revealing any information.

5.1 Certificate Authority Setup

We created a minimal PKI using OpenSSL command-line tools via two shell scripts:

1. `setup_ca.sh` — Generates a 4096-bit RSA private key (`ca.key`) and a self-signed root certificate (`ca.crt`) for the CA. The CA subject is `/C=IN/O=CS6008 SecureChat/CN=SecureChat Root CA`.

2. `issue_server_cert.sh <server_ip>` — Generates a 2048-bit RSA key pair for the server, creates a Certificate Signing Request (CSR) with the server's IP as the Common Name (CN), and signs it with the CA's private key to produce `server.crt`.

The key commands used are:

```
# CA setup
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 \
    -days 1 -subj "/C=IN/O=CS6008 SecureChat/CN=SecureChat Root
        CA" \
    -out ca.crt

# Server certificate issuance
openssl genrsa -out server.key 2048
openssl req -new -key server.key \
    -subj "/C=IN/O=CS6008 SecureChat/CN=<server_ip>" \
    -out server.csr
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -days 1 -sha256 \
    -extfile server_ext.cnf -out server.crt

# Verify the chain
openssl verify -CAfile ca.crt server.crt
```

Distribution:

- `ca.crt` → copied to every Client VM (trusted root).
- `server.key`, `server.crt` → stay on the Server VM only.
- `ca.key` → stays on the Server VM, never distributed.

5.2 Updated Handshake Protocol

The Phase 3 handshake occurs *before* any DH exchange or application data:

From → To	Message
Server → Client	`CERT <base64-encoded PEM certificate>`
Client validates	Signature, validity dates, CN match
Client → Server	`CHALLENGE <32-byte random hex nonce>`
Server → Client	`SIG <base64-encoded RSA-SHA256 signature>`
Client verifies	Signature against cert's public key
Client → Server	`DH_INIT <hex` $g^a \bmod p$`>`
Server → Client	`DH_ACK <hex` $g^b \bmod p$`>`

If *any* validation check fails, the client aborts immediately – it does not send a challenge, proceed with DH, or reveal any further information.

5.3 Certificate Validation (Client-Side)

The client performs three checks in order (implemented in `pki.cpp`, function `validate_certificate`):

1. **Signature check** — Verifies that the received certificate was actually signed by the trusted CA's key using `X509_verify()`.
2. **Validity period check** — Confirms the current time falls within the certificate's `notBefore`/`notAfter` window using `X509_cmp_current_time()`.
3. **Identity (CN) check** — Extracts the certificate's Common Name and verifies it exactly matches the expected server address the client was configured with.

A `std::runtime_error` is thrown with a specific, distinguishable reason on the *first* check that fails, and the handshake aborts.

5.4 Proof-of-Possession (Challenge–Response)

After certificate validation passes, the client sends a random 32-byte challenge nonce. The server signs this nonce with its private key using RSA + SHA-256 (`EVP_DigestSign`). The client verifies the signature using the public key embedded in the *same certificate it just validated*, confirming that the server truly holds the matching private key – not merely a copy of the certificate file.

5.5 Mallory's Setup for MITM Re-attempt

A script `mallory/generate_fake_certificate.sh` generates Mallory's own self-signed certificate and key pair:

```
openssl genrsa -out fake.key 2048
openssl req -x509 -new -nodes -key fake.key -sha256 -days 365 \
    -subj "/C=XX/O=Mallory Attacker/CN=<spoofed_server_ip>" \
    -out fake.crt
```

Mallory's certificate has the *correct CN* (same as the real server) but is **not signed by the trusted CA**. This demonstrates that CN spoofing alone is insufficient – the CA signature check is what actually stops the attack.

The MITM proxy (`mitm.cpp`) listens on port 5001 and attempts two things:

1. **Pose as server** to the victim client – sends Mallory's fake certificate.
2. **Pose as client** to the real server – performs a legitimate handshake.

5.6 Evidence and Results

5.6.1 Server Logs – Legitimate Flow

Figure 11 shows the server-side logs. Bob connected successfully: the server sent its CA-signed certificate, responded to the client's challenge with a valid signature, and completed the DH key exchange, with the session established using matching key fingerprints.

When Alice's connection was routed through Mallory, the server either saw Mallory's connection attempt as a separate client or did not receive Alice's connection at all – confirming that the MITM proxy intercepted Alice's traffic before it could reach the real server.

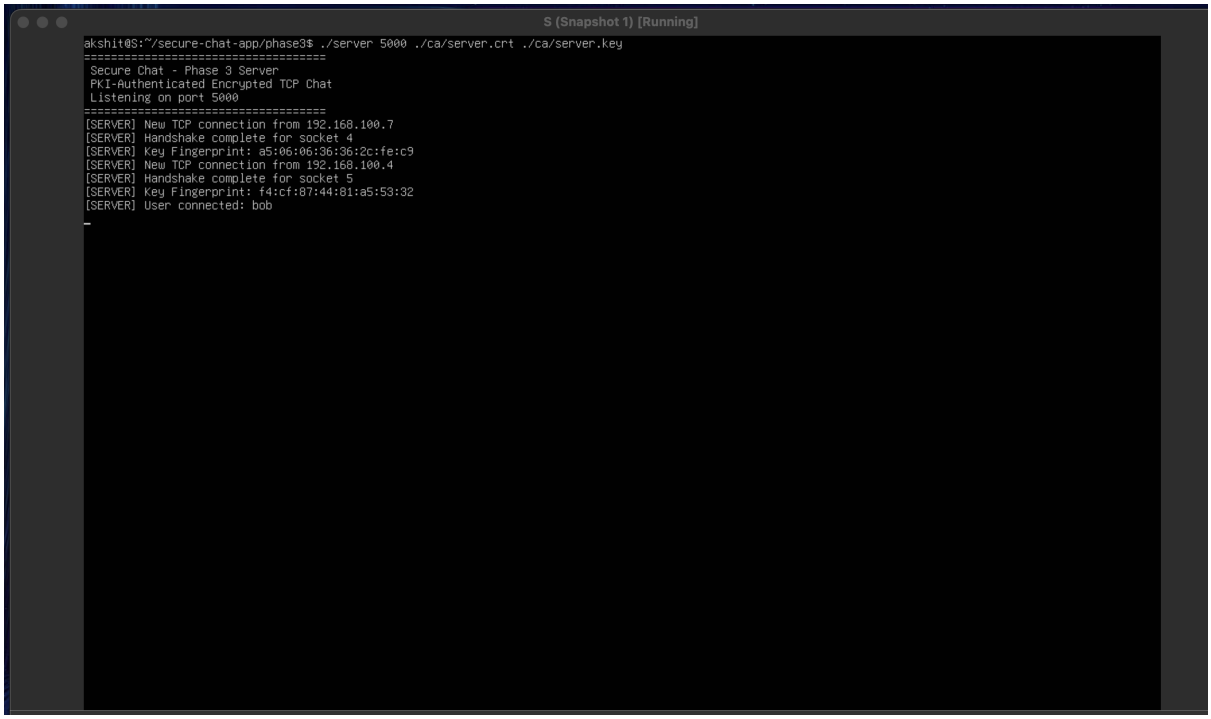

Figure 11: Server logs showing successful handshake with Bob and the connection behaviour when Mallory intercepts Alice's traffic.

5.6.2 Client Logs – Alice (Blocked) and Bob (Successful)

Figure 12 shows both clients. **Bob** connected directly to the real server, validated the CA-signed certificate, verified the server's proof-of-possession, and established a secure session with matching DH fingerprints, demonstrating that the legitimate flow works correctly.

Alice, whose connection was routed through Mallory's proxy, received Mallory's self-signed certificate. The client's certificate validation immediately detected that the certificate was **not signed by the trusted CA** and aborted the connection. Alice did not send a challenge, did not proceed with DH, and revealed no information – exactly as specified by the Phase 3 requirements.

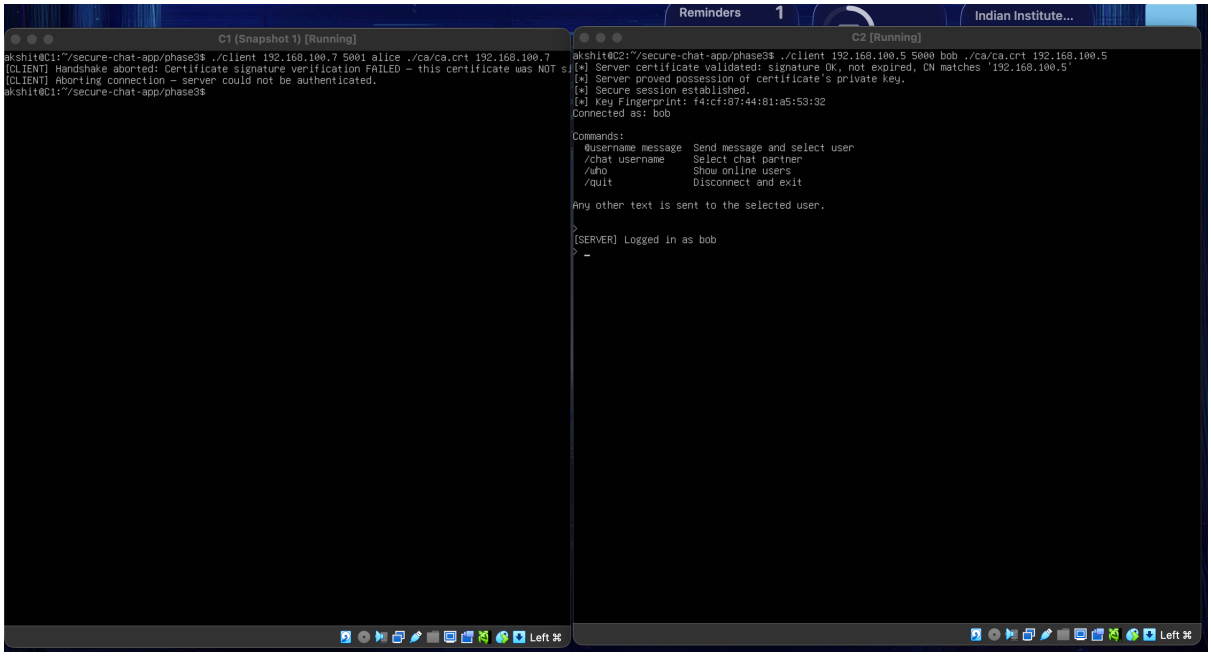

Figure 12: Client terminals: Alice's connection to Mallory is rejected (certificate verification failure); Bob's connection to the real server succeeds with full certificate and proof-of-possession validation.

5.6.3 Mallory's MITM Proxy Logs

Figure 13 shows the MITM proxy output. Mallory sent her self-signed certificate to Alice, but Alice immediately closed the connection without sending a `CHALLENGE` message. The attack summary printed by the proxy confirms:

- **Pose as SERVER to victim**: `FAILED` (expected – Phase 3 defense holds).
- **Pose as CLIENT to server**: `SUCCEEDED` (expected – the server has no client-authentication requirement in this phase).

Since the server-impersonation half failed, the full MITM relay could not be established. This is a direct contrast with Phase 2, where the same MITM attack succeeded completely and Mallory could intercept all plaintext.

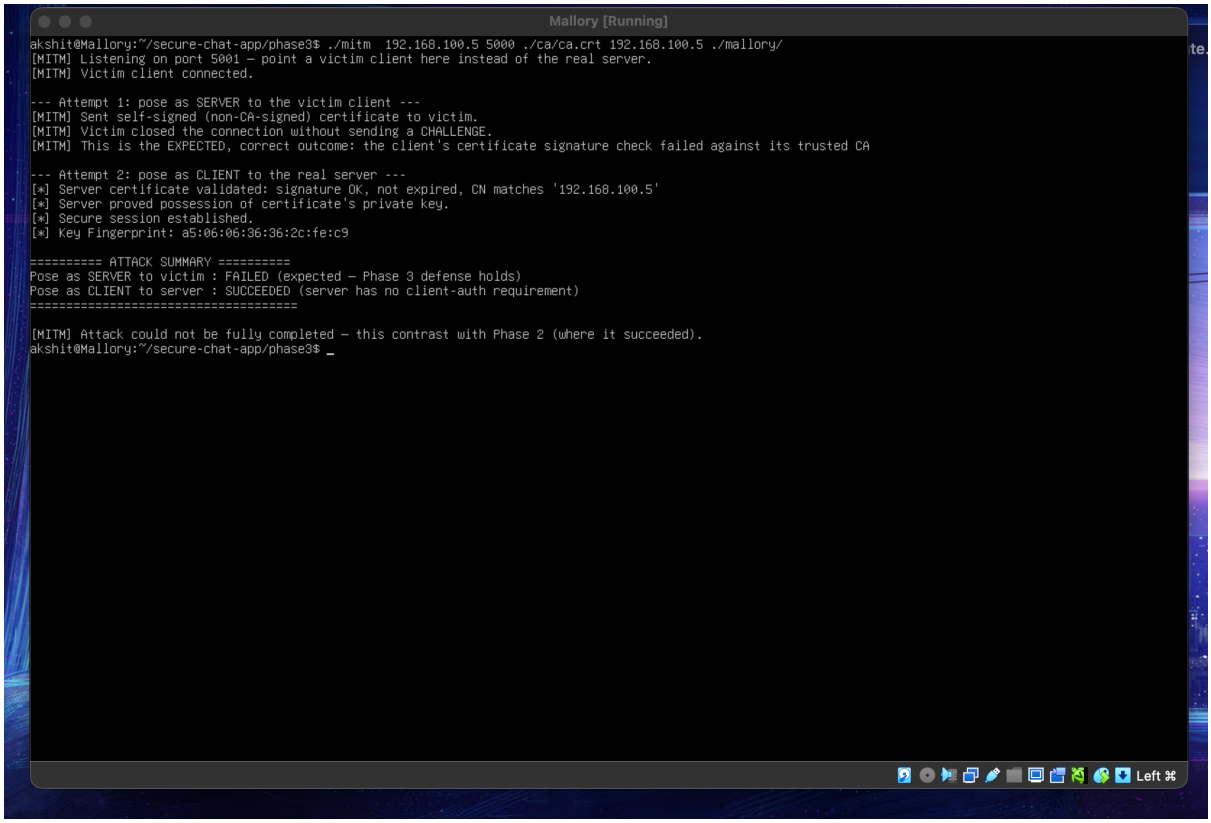

Figure 13: Mallory's MITM proxy logs showing the attack failed — the victim client rejected the fake certificate and disconnected before any secrets were exchanged.

5.7 Phase 2 vs. Phase 3: Contrast

Aspect	Phase 2	Phase 3
Server identity verification	None	CA-signed certificate + CN check
Proof-of-possession	None	Challenge–response signature
MITM: pose as server to client	**Succeeded**	**Failed**
MITM: full relay with plaintext	**Succeeded**	**Failed**
Client reveals data to attacker	Yes (password, DH)	No (aborts immediately)

In Phase 2, Mallory could perform two independent DH exchanges – one with the client and one with the server – and relay/read all plaintext. In Phase 3, the client verifies the server's certificate before proceeding with any key exchange. Since Mallory does not possess the CA's private key, she cannot produce a valid certificate, and the attack is defeated at the very first step.

5.8 Source Files

File	Purpose
`pki.h / pki.cpp`	Certificate loading, validation, signing, and verification
`handshake.h / handshake.cpp`	Updated handshake with CERT/CHALLENGE/SIG ste
`openssl_raii.h`	RAII wrappers for OpenSSL objects (X509, EVP_PKEY
`server.cpp`	Loads server cert/key, calls server handshake
`client.cpp`	Loads CA cert, calls client handshake with CN check
`mitm.cpp`	MITM re-attempt: uses fake self-signed cert
`ca/setup_ca.sh`	Creates the Certificate Authority
`ca/issue_server_cert.sh`	Issues the server certificate
`mallory/generate_fake_certificate.sh`	Generates Mallory's self-signed fake cert

5.9 Conclusion

Phase 3 successfully mitigates the MITM vulnerability demonstrated in Phase 2. By requiring the server to present a CA-signed certificate and prove possession of the corresponding private key *before* any Diffie–Hellman exchange, the client can distinguish the legitimate server from an impersonator. Mallory's attack, which succeeded completely in Phase 2, is now defeated at the certificate validation step – the client aborts without exposing any secrets.

6 Phase 4: End-to-End Encryption Between Clients

In Phases 2 and 3 we secured the link between each client and the server using Diffie–Hellman key exchange and AES-256-GCM encryption, and authenticated the server via PKI. However, the server itself can still read every message it relays – it decrypts incoming traffic from one client and re-encrypts it for the other.

Phase 4 adds an **end-to-end (E2E) encryption layer** between two clients (C1 and C2). After an E2E session is established, chat messages carry an inner layer of AES-256-GCM encryption using a key that only the two clients know. The server continues to relay the messages, but it sees only opaque ciphertext – it cannot derive the C1 $\leftrightarrow$ C2 key or read the content.

6.1 Client-to-Client Key Exchange

When a user types `/e2e <username>`, the client initiates a Diffie–Hellman key exchange *with the peer*, routed through the server as ordinary messages. The server's existing relay logic does not need to change – it just forwards opaque data by username.

The exchange uses the same DH implementation from Phase 2 (2048-bit MODP Group 14, hand-coded modular exponentiation with OpenSSL `BN_mod_exp`). The resulting shared secret is hashed through SHA-256 to produce a 256-bit AES key, just like the client–server key.

6.2 Wire-Level Tagging

To distinguish handshake messages from encrypted chat, we use the wire tags required by the assignment specification:

Tag	Purpose
__E2E_INIT__<data>	Initiator sends its DH public value to the peer
__E2E_ACK__<data>	Responder replies with its DH public value
__E2E_MSG__<data>	Inner-encrypted chat message (AES-256-GCM ciphertext)

These tags are prepended to the message payload inside the existing `MSG <recipient> <payload>` format. The server sees these tags as part of the message body but does not interpret them – it relays them as-is.

6.3 Double Encryption

Once the E2E session is active, every chat message goes through two layers of encryption:

1. **Inner layer (E2E key):** The plaintext message is encrypted with AES-256-GCM using the C1 ↔ C2 shared key, base64-encoded, and prepended with `__E2E_MSG__`.
2. **Outer layer (client–server key):** The tagged payload is wrapped inside `MSG <recipient> ...` and encrypted again with AES-256-GCM using the client–server session key from Phase 3.

The server decrypts the outer layer (to route the message), but the inner blob remains encrypted with a key the server never had.

6.4 Simultaneous Initiation Handling

If both clients happen to type `/e2e` for each other at roughly the same time, a race condition could occur. Our implementation handles this by reusing the already-generated DH keypair when a peer's `__E2E_INIT__` arrives while we have a pending handshake for that same peer. Since DH is symmetric ($g^{ab} \bmod p = g^{ba} \bmod p$), both sides converge on the same shared secret without any extra tie-breaking logic.

6.5 Message Flow

The overall flow when Alice sends "hello" to Bob with E2E active:

```
Alice encrypts "hello" with E2E key      -> inner ciphertext
Alice prepends __E2E_MSG__ tag           -> __E2E_MSG__<base64 blob>
Alice wraps as MSG bob __E2E_MSG__...    -> application-layer message
Alice encrypts with client-server key    -> outer ciphertext (sent on wire)

Server decrypts outer layer              -> sees: MSG bob __E2E_MSG__<opaque>
Server logs: "alice -> bob: __E2E_MSG__<opaque>"  (cannot read content)
Server re-encrypts for Bob's link        -> delivered to Bob

```

```
Bob decrypts outer layer                    -> FROM alice
    __E2E_MSG__<base64 blob>
Bob strips tag, decrypts inner blob         -> "hello"
```

6.6 Evidence and Results

6.6.1 Alice and Bob – E2E Session and Chat

Figure 14 shows the terminals of Alice and Bob:

- Both clients connect to the server and complete the Phase 3 handshake (certificate validation + proof-of-possession + DH).
- One client runs `/e2e <peer>`, which triggers the client-to-client DH exchange via `__E2E_INIT__` and `__E2E_ACK__`.
- Both clients print the **E2E Key Fingerprint** – these fingerprints **match** on both sides, confirming that they derived the same shared secret independently, without any involvement of the server.
- After the session is established, messages display with the `(E2E)` label, confirming they were decrypted using the end-to-end key.
- Chat works correctly in both directions – messages sent by Alice are received and correctly decrypted by Bob, and vice versa.

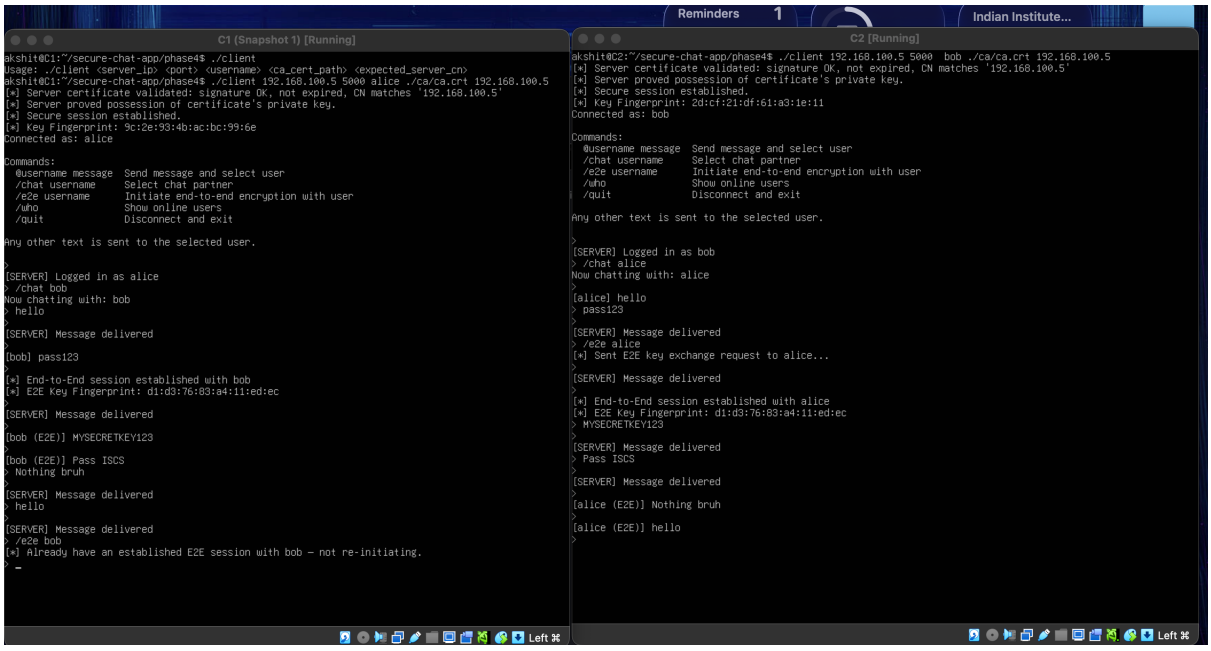

Figure 14: Alice and Bob terminals showing the E2E key exchange, matching fingerprints on both sides, and encrypted chat with the (E2E) label.

6.6.2 Server Logs – Server Blindness

Figure 15 shows the server's relay log. The key observation is the contrast in what the server can see:

- **Before E2E:** If a message was sent before the E2E session was established, the server log would show the plaintext content (e.g., `[RELAY PLAINTEXT] alice -> bob: hello`).
- **After E2E:** Once the E2E session is active, the server log shows only opaque data such as `[RELAY PLAINTEXT] alice -> bob: __E2E_MSG__<base64 blob>`. The server can see *who* is talking to *whom* (routing metadata), but the actual message content is unreadable ciphertext.
- The E2E handshake messages (`__E2E_INIT__`, `__E2E_ACK__`) also appear in the server log as opaque DH public values – the server has no way to derive the shared secret from these.

This confirms that the server acts as a **blind relay** for E2E traffic – it can route messages by username but cannot decrypt or read any of the chat content.

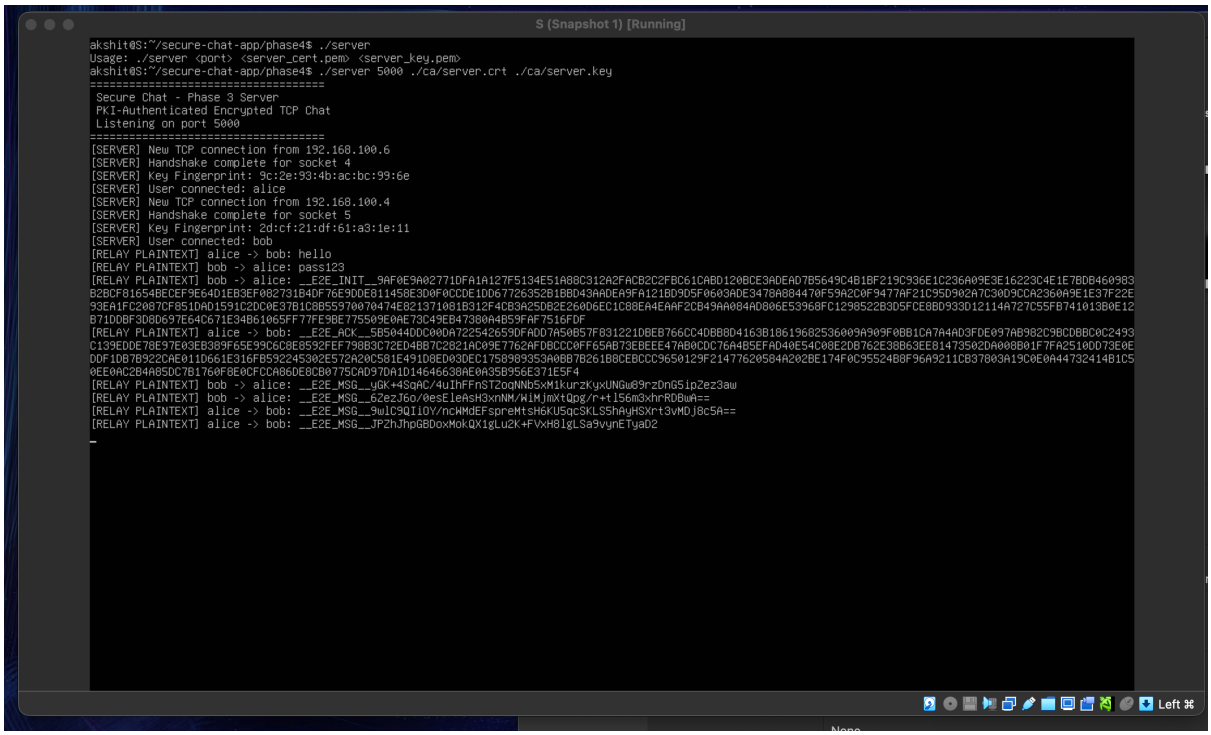

Figure 15: Server relay log showing that after E2E is established, the server only sees opaque tagged data (`__E2E_MSG__<ciphertext>`) instead of readable plaintext.

6.7 Key Differences from Phase 3

Aspect	Phase 3	Phase 4
Client–server encryption	Yes (AES-256-GCM)	Yes (unchanged)
Server can read messages	Yes	No (after E2E)
Client-to-client shared key	None	Independent DH
Encryption layers per message	1 (outer only)	2 (inner + outer)
Server code changes needed	—	None (relay is tag-agnostic)

6.8 Source Files Modified

File	Changes
`client.cpp`	Added E2E key exchange (`/e2e` command), wire tag handling, double encryption/decryption, simultaneous initiation handling, per-peer key storage
`server.cpp`	Added `[RELAY PLAINTEXT]` logging to show what the server sees (no crypto changes needed)

All other files (`dh.cpp`, `aes_gcm.cpp`, `pki.cpp`, `handshake.cpp`, etc.) are reused from Phase 3 without modification. The server's relay logic required zero changes – it continues to forward `MSG` payloads by username, unaware of the inner encryption.

6.9 Conclusion

Phase 4 successfully adds end-to-end encryption between clients. The two clients perform their own Diffie–Hellman key exchange through the server, derive a shared secret that the server never sees, and wrap every subsequent chat message in an additional layer of AES-256-GCM encryption. The matching fingerprints on both clients confirm key agreement, and the server logs confirm that it can only see opaque ciphertext after E2E is established.

7 Phase 5: Forward Secrecy via Periodic Key Rotation

In Phase 4 we established end-to-end encryption between two clients using a single Diffie–Hellman shared key. That key stays the same for the entire session. If an attacker records all the encrypted traffic and later manages to compromise that one key (e.g., through memory forensics on a client machine), they can go back and decrypt *every* message from the entire session.

Phase 5 addresses this by adding **forward secrecy**. The E2E key is automatically renegotiated every 60 seconds. Each rotation produces a completely new key through a fresh DH exchange, and the old key is discarded. Even if an attacker compromises one key, they can only decrypt messages from that single 60-second window – all earlier (and later) messages remain protected.

7.1 Automatic Key Rotation Timer

A background thread (`e2e_timer_thread`) runs alongside the main client. Every second, it checks whether any active E2E session has been using its current key for 60 seconds or more. When the timer fires, it triggers a rekey by sending a fresh DH public value to the peer.

```
// Simplified logic from the timer thread
for each active E2E session:
    elapsed = now - session.last_rotation
    if elapsed >= 60 seconds:
        trigger_rekey(peer)
```

7.2 Rekey Handshake

The rekey uses two new wire-level tags, separate from the initial E2E handshake:

Tag	Purpose
`__E2E_REKEY_INIT__<data>`	New DH public value from the initiator
`__E2E_REKEY_ACK__<data>`	New DH public value from the responder

The flow is straightforward:

1. The timer fires on one client. It generates a fresh DH keypair and sends `__E2E_REKEY_INIT__` with its new public value.
2. The peer receives it, generates its own fresh DH keypair, computes the new shared secret, and replies with `__E2E_REKEY_ACK__`.
3. Both sides now have a new key. The old key is kept briefly as `previous_key` (to decrypt any messages still in transit), then discarded on the next rotation.

7.3 Simultaneous Rekey Collision Handling

Since both clients run independent 60-second timers, they might both try to initiate a rekey at roughly the same time. We handle this with a simple **lexicographic tie-breaker**:

- When a client receives a `REKEY_INIT` while it already has its own `REKEY_INIT` pending for the same peer, it compares usernames.
- The client with the **lexicographically smaller** username wins – it ignores the peer's `REKEY_INIT` and waits for its own to be acknowledged.
- The other client drops its own pending attempt and responds to the peer's instead.

This ensures both sides always converge on exactly one new key, with no extra round trips or risk of ending up with different keys.

7.4 Graceful Transition During Rotation

Messages sent right before or during a rotation might be encrypted with the old key but arrive after the new key is active. To handle this without dropping any messages, each session keeps a short-lived `previous_key`. On decryption, the client first tries the current key; if that fails, it falls back to the previous key:

```
try:
    plaintext = decrypt(current_key, ciphertext)
catch:
    plaintext = decrypt(previous_key, ciphertext)
```

This guarantees that no messages are lost or become unreadable during a rotation.

7.5 Stale Rekey Timeout

If a `REKEY_INIT` is sent but the peer never responds (e.g., they disconnected), the pending entry is automatically cleared after 10 seconds so the timer can retry on the next cycle instead of being stuck forever.

7.6 Evidence and Results

7.6.1 Alice and Bob – Key Rotation Logs

Figure 16 shows the terminals of Alice and Bob over a session lasting more than two minutes:

- After the initial E2E session is established, both clients print the same **E2E Key Fingerprint**, confirming they share the same key.
- Every 60 seconds, the rotation triggers automatically. Both clients print `[*] Session Key Rotated with <peer>. New Fingerprint: <hash>`.
- The fingerprint **changes with every rotation** – each new key is genuinely different from the previous one.
- After each rotation, both clients show the **same new fingerprint**, confirming they agreed on the same key.
- Messages sent **immediately after a rotation** are correctly decrypted and displayed with the `(E2E)` label, showing that the rotation did not break the session.

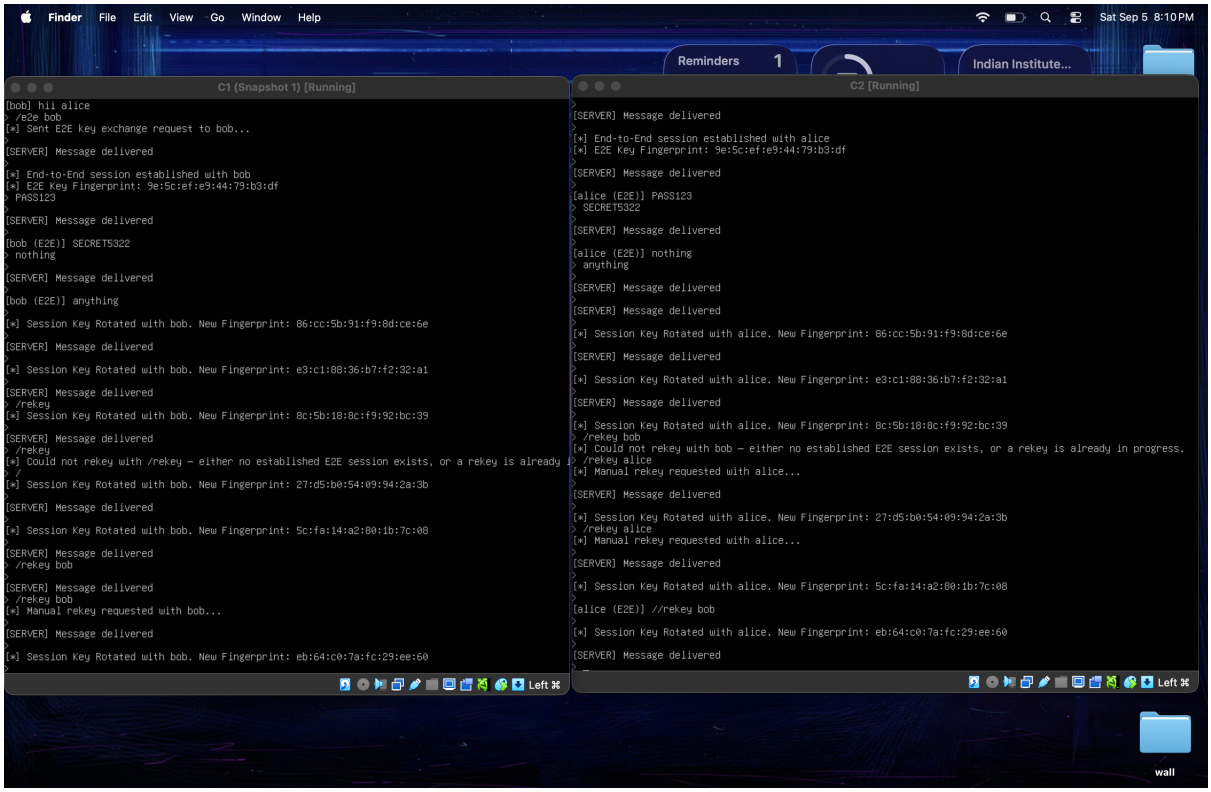

Figure 16: Alice and Bob terminals showing multiple key rotations every 60 seconds. Fingerprints change each time and match on both sides. Chat continues to work across rotations.

7.6.2 Server Logs

Figure 17 shows the server's relay log during the same session. The server sees:

- The `__E2E_REKEY_INIT__` and `__E2E_REKEY_ACK__` messages as opaque DH public values – it cannot derive the new shared secret.
- All chat messages after E2E as `__E2E_MSG__<ciphertext>` – unreadable to the server, regardless of which rotation key was used.

The server remains a blind relay throughout. It does not need any code changes for Phase 5 – the rekey tags are just more opaque payloads it forwards by username.

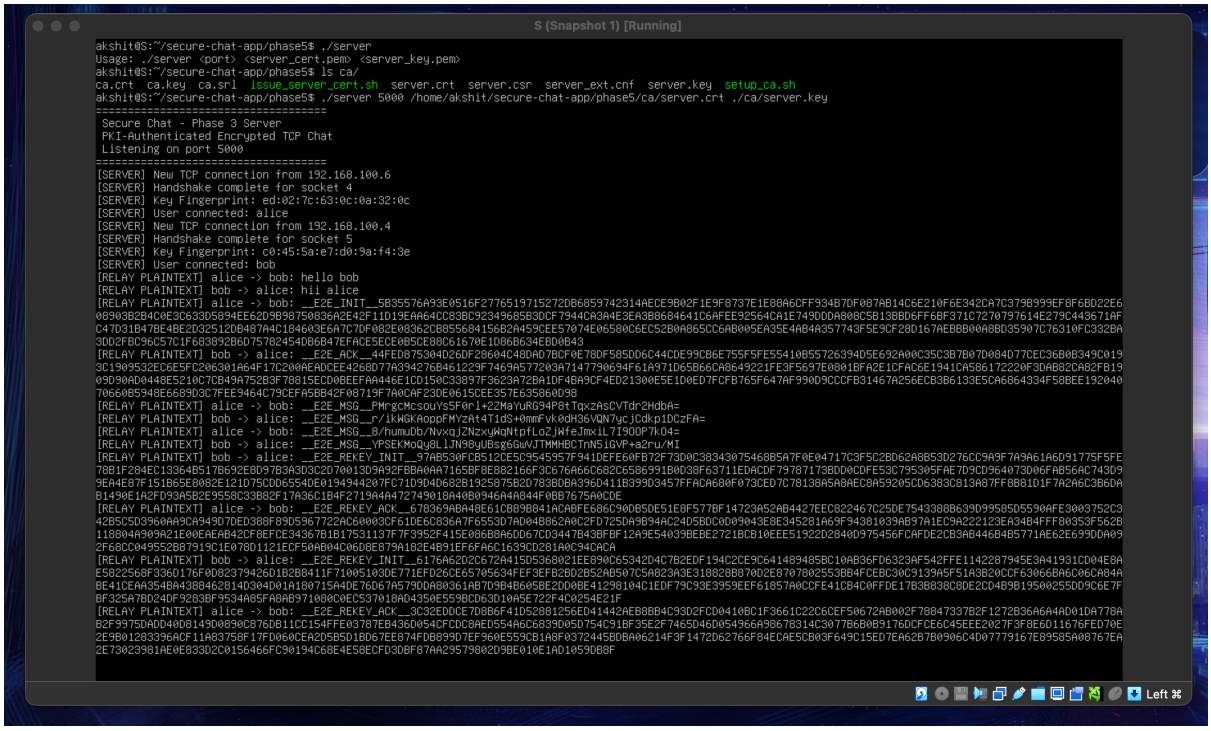

Figure 17: Server relay log showing opaque rekey handshake messages and encrypted chat content. The server cannot read any of it.

7.7 What Forward Secrecy Buys Over Phase 4

In Phase 4, a single long-lived key protects the entire session. If an attacker later compromises that key – through a memory dump, a side-channel attack, or coercion – they can decrypt **every message** from the entire conversation, past and future.

With Phase 5's periodic rotation:

- Each key is used for at most 60 seconds. A compromised key exposes only that one 60-second window of messages.
- Old keys are discarded after rotation. There is no stored material an attacker can use to reconstruct past keys – each key comes from a fresh DH exchange with new random exponents.
- Future keys are also safe. Since each rotation generates independent random DH parameters, knowing one key gives no information about the next.
- In concrete terms: an attacker who records a week of encrypted traffic and then compromises a single key can decrypt at most 60 seconds of that traffic. The remaining days remain encrypted with keys that no longer exist anywhere.

Scenario	**Phase 4**	**Phase 5**
One key compromised	All messages exposed	Only 60s of messages
Old keys recoverable	N/A (one key for all)	No (discarded after use)
Future keys derivable from old	N/A	No (fresh DH each time)
Session disrupted by rotation	N/A	No (graceful fallback)

7.8 Source Files Modified

Only `client.cpp` was modified from Phase 4. No server-side changes were needed.

Change	Details
`E2ESession` struct	Added `previous_key`, `last_rotation`, `has_previous` fields
`PendingRekey` struct	Tracks in-flight rekey DH + timestamp
`e2e_timer_thread()`	Background thread checking 60s expiry
`trigger_rekey()`	Shared logic for auto and manual rekey
Rekey handlers	`REKEY_INIT` / `REKEY_ACK` processing with collision tie-breaker
Decryption fallback	Try current key, then previous key
`/rekey` command	Manual rekey trigger for testing
Stale timeout	10s timeout on unanswered rekey attempts

7.9 Conclusion

Phase 5 adds forward secrecy to the end-to-end encrypted session from Phase 4. The key is automatically rotated every 60 seconds through a fresh Diffie–Hellman exchange, old keys are discarded, and a lexicographic tie-breaker handles simultaneous rotation attempts. The screenshots confirm that fingerprints change with every rotation, both sides always agree on the new key, and chat continues to work across rotations without any disruption.

8 Overall Conclusion

Across the five phases, we built a chat application that evolved from a fully open, plaintext relay into a system with confidentiality, server authentication, end-to-end encryption, and forward secrecy. Phase 1 established a working baseline with no protection at all. Phase 2 added confidentiality on each client–server link through a hand-implemented Diffie–Hellman exchange and AES-256-GCM, but a self-built MITM proxy showed that key exchange without authentication is not enough – Mallory could intercept everything without either side noticing. Phase 3 closed that gap with a minimal PKI and a proof-of-possession challenge, which caused the same MITM attack to fail at the very first step. Phase 4 removed the server from the trust boundary entirely by giving the two clients their own independent shared key, so the server could only see opaque ciphertext. Finally, Phase 5 limited the impact of any future key compromise by rotating the end-to-end key every 60 seconds, so that a single leaked key exposes at most one minute of conversation.

Each phase was verified with fingerprint matching, Wireshark captures, and terminal logs from the server and both clients, and each cryptographic addition was tested against an attack built specifically to break the previous phase.